## TITLE:

CRYPT ARITHMETIC PROBLEM

## OBJECTIVES: .

- To understand the concept of cryptarithmetic problems.
- To represent letters as digits in arithmetic problems using Prolog.
- To apply logic and constraints to find unique solutions.
- To learn how Prolog handles backtracking and variable assignments.

## THEORY:

Cryptarithmetic problems are puzzles where letters represent digits. Each letter must be assigned a unique digit (0–9) such that the arithmetic operation is valid. Prolog, a logic programming language, is well-suited for solving such problems using facts, rules, and arithmetic constraints. The program generates all possible digit assignments and automatically checks which satisfy the equation, ensuring uniqueness and correctness.

## PROGRAM CODE:

1) Solve the problem for the following arithmetic problem.

i) LETS + WAVE =LATER

```
solution(Z) :-
digit(L), L>0,
digit(E),
digit(T),
digit(S),
digit(W), W>0,
digit(A),
digit(V),
digit(R),
1000*L + 100*E + 10*T + S +
1000*W + 100*A + 10*V + E =:=
10000*L + 1000*A + 100*T + 10*E + R,
Z = [L,E,T,S,W,A,V,R], different(Z).
digit(0).  digit(1).  digit(2).  digit(3).
digit(4).  digit(5).  digit(6).  digit(7).
digit(8).  digit(9).
```

OUTPUT :

```
1 ?-  solution(Z).
Z = [1, 5, 6, 7, 9, 0, 8, 2]
```

b) SEND + MORE = MONEY

```
solution(L):-digit(S),S>0,digit(E),digit(N),digit(D),
digit(M),M>0,digit(O),digit(R),digit(Y),
1000*S+100*E+10*N+D+
1000*M+100*O+10*R+E=:=
10000*M+1000*O+100*N+10*E+Y,
L=[S,E,N,D,M,O,R,Y],
different(L). digit(0). digit(1).
```

```
digit(2). digit(3). digit(4).
digit(5). digit(6). digit(7).
digit(8). digit(9). different([]).
different([X|R]):-not(member(X,R)),different(R).
```

OUTPUT :

```
1 ?- solution(Z).
Z = [9, 5, 6, 7, 1, 0, 8, 2]
```

c) TWO + TWO = FOUR

```
solution(Z) :-
digit(T), T>0,
digit(W),
digit(O),
digit(F), F>0,
digit(U),
digit(R),
(100*T + 10*W + O) * 2 =:=
1000*F + 100*O + 10*U + R,
Z = [T,W,O,F,U,R],
different(Z). digit(0).
digit(1). digit(2). digit(3).
digit(4). digit(5). digit(6).
digit(7). digit(8). digit(9).
different([]). different([X|P])
:- not(member(X,P)),
different(P).
```

OUTPUT

:

```
1 ?- solution(Z).
Z = [7, 3, 4, 1, 6, 8]
```

d) IT + IS = ME solution(L) :- digit(I), I>0, digit(T), digit(S), digit(M), M>0, digit(E), 10*I + T +

```
10*I + S =:=
10*M + E,
L = [I,T,S,M,E],
different(L).
digit(0). digit(1).
digit(2). digit(3).
digit(4). digit(5).
digit(6). digit(7).
digit(8). digit(9).
different([]).
different([X|P]) :-
not(member(X,P))
, different(P).
```

OUTPUT :

```
1 ?-  solution(Z).
Z = [1, 2, 8, 3, 0]
```

e) BASE + BALL = GAMES

```
solution(Solution)
:- digit(B), B > 0,
digit(A), digit(S),
digit(E), digit(L),
digit(G), G > 0,
digit(M),
1000*B + 100*A + 10*S + E +
1000*B + 100*A + 10*L + L =:=
10000*G + 1000*A + 100*M + 10*E + S,
Solution =
[B,A,S,E,L,G,M],
different(Solution).
digit(0).  digit(1).  digit(2).
digit(3).  digit(4).  digit(5).
digit(6).  digit(7).  digit(8).
digit(9).  different([]).
different([X|P]) :-
not(member(X,P)),
different(P).
```

OUTPUT :

```
1 ?-  solution(Solution).
Solution = [7, 4, 8, 3, 5, 1, 9]
```

## DISCUSSION AND CONCLUSION:

In this lab, cryptarithmetic problems were solved using Prolog by representing letters as digits and applying constraints. Prolog used backtracking and uniqueness checks to find valid solutions for each puzzle, demonstrating how logic and arithmetic constraints were managed effectively. The lab helped understand logical reasoning, constraint satisfaction, and the practical application of Prolog in solving arithmetic puzzles.

since 2001

# COSMOS COLLEGE OF MANAGEMENT AND TECHNOLOGY

Affiliated to Pokhara University

LAB REPORT NO : 02

LAB REPORT ON :

CRYPT ARITHMETIC PROBLEM


SUMMITTED BY:

Name: Oshan Bajracharya

Roll No: 230328

Faculty: BE-Computer

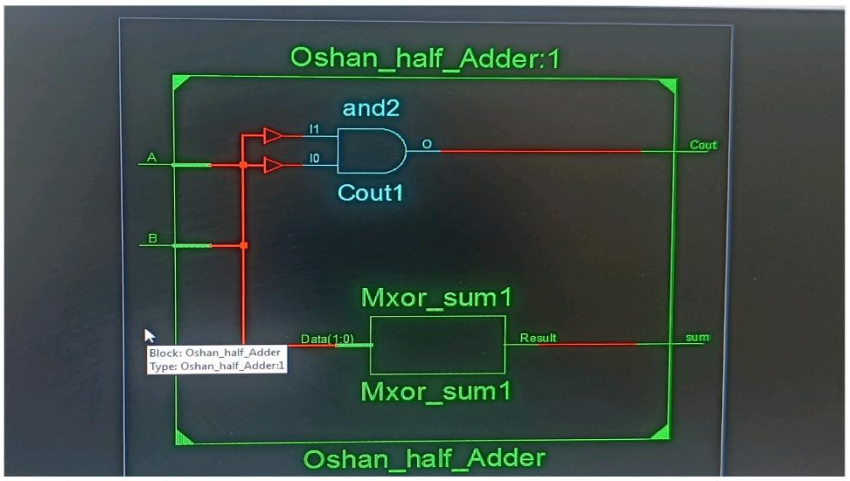
Group: B

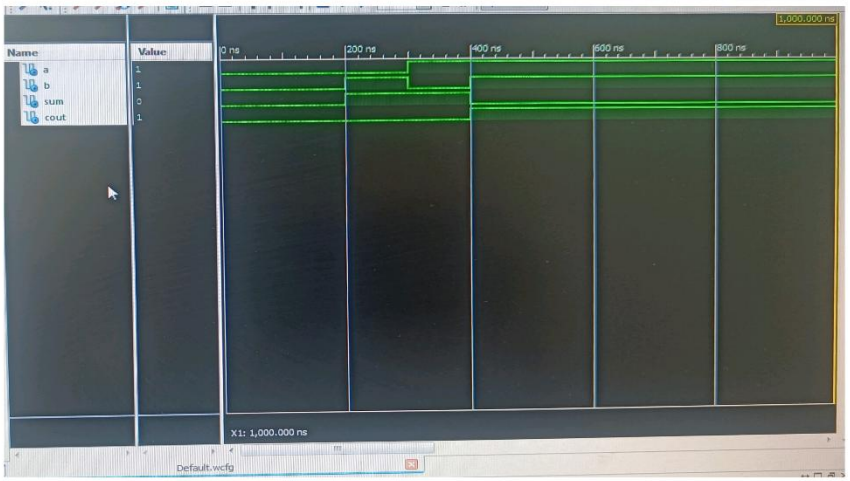
SUMMITTED TO:

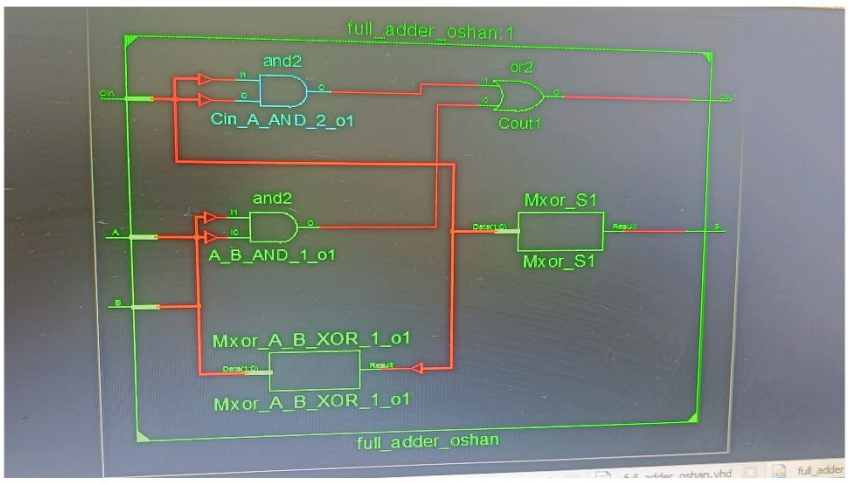
Department Of ICT

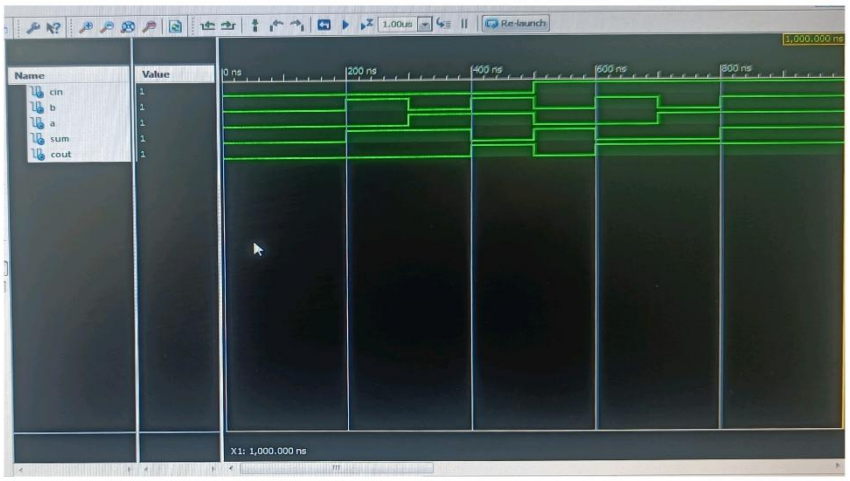

Date of Experiment:

Date of Submission:


Oshan_half_Adder:1
and2
I1
I0
O
A
B
Cout
Cout1
Mxor_sum1
Data[1:0]
Result
sum
Block: Oshan_half_Adder
Type: Oshan_half_Adder:1
Mxor_sum1
Oshan_half_Adder


Name
Value
a
b
sum
cout
0 ns
200 ns
400 ns
600 ns
800 ns
1,000.000 ns
X1: 1,000.000 ns
Default.wcfg


full_adder_oshan:1
and2
or2
Cin
Cin_A_AND_2_o1
Cout1
and2
Mxor_S1
A
A_B_AND_1_o1
Mxor_S1
B
Mxor_A_B_XOR_1_o1
Mxor_A_B_XOR_1_o1
full_adder_oshan


Re-launch
Name
Value
cin
b
a
sum
cout
0 ns
200 ns
400 ns
600 ns
800 ns
1,000.000 ns
X1: 1,000.000 ns